

SEGURANÇA EM PIPELINE: PROTEÇÃO DE DADOS ESPACIAIS, AUTOMAÇÃO E RESILIÊNCIA OPERACIONAL

Aplicação de práticas DevSecOps em uma plataforma de monitoramento socioambiental baseada em dados espaciais.

Resumo. Este trabalho apresenta a integração de práticas de DevSecOps ao EcoSaúde, solução da Global Solution voltada ao monitoramento socioambiental por meio de dados espaciais, inteligência artificial e computação de borda. O objetivo é proteger dados sensíveis, APIs críticas, infraestrutura em nuvem, contêineres e componentes de orquestração desde o início do desenvolvimento.

A proposta considera que o EcoSaúde não opera apenas como uma aplicação comum. A plataforma depende de dados ambientais, geoespaciais e de telemetria para gerar previsões que podem apoiar decisões de saúde pública, saneamento e sustentabilidade. Por isso, uma falha de segurança pode comprometer não apenas servidores ou credenciais, mas também a confiança dos indicadores apresentados à comunidade e aos gestores públicos.

1. Contextualização da segurança no EcoSaúde

O EcoSaúde utiliza dados provenientes de satélites, APIs climáticas, bases ambientais e serviços geoespaciais para identificar vulnerabilidades em regiões monitoradas. A plataforma possui microsserviços dedicados a forecast, saúde pública, saneamento, qualidade da água, qualidade do ar e análise geoespacial. Esses serviços são empacotados em contêineres e preparados para execução em ambiente orquestrado, como Kubernetes ou K3s.

Nesse cenário, a segurança precisa ser incorporada ao fluxo de desenvolvimento. Se uma credencial for exposta no repositório, um atacante pode consumir APIs externas em nome da aplicação. Se uma dependência vulnerável for implantada, a API pode ser explorada. Se um contêiner for executado com privilégios excessivos, o impacto de uma falha aumenta. Por isso, o módulo DevSecOps foi estruturado para validar automaticamente código, dependências, segredos, imagens e manifestos antes do deploy.

2. Fundamentação do risco

A preocupação com segurança é reforçada pelo impacto social do projeto. Em 2024, a Organização Mundial da Saúde registrou mais de 14,4 milhões de casos de dengue no mundo, com mais de 11 mil mortes. Esse cenário mostra a importância de ferramentas de previsão e alerta. Ao mesmo tempo, dados do relatório WHO/UNICEF JMP de 2025 indicam que 2,1 bilhões de pessoas ainda não possuem acesso a água potável gerenciada com segurança. Portanto, sistemas que apoiam saúde, água e saneamento precisam operar com alta confiabilidade.

No EcoSaúde, a integridade dos dados é tão importante quanto a disponibilidade do sistema. Um forecast gerado com base em dados adulterados, incompletos ou processados por componentes vulneráveis pode induzir decisões equivocadas. Por isso, a segurança do pipeline foi tratada como uma camada de governança técnica da própria solução.

3. Mapeamento de riscos no pipeline

Risco	Cenário no EcoSaúde	Impacto	Controle
Vazamento de segredos	Chaves de APIs satelitais, climáticas, mapas e banco são incluídas no código ou em arquivos .env.	Acesso indevido a serviços externos, custos indevidos e exposição de dados.	GitHub Secrets, Gitleaks e bloqueio de pipeline.
Dependências vulneráveis	Bibliotecas Python com CVEs conhecidas são usadas pelos microsserviços.	Exploração da API e comprometimento de dados usados no forecast.	pip-audit e Dependabot.
Imagem Docker insegura	Imagem desatualizada, excesso de pacotes ou execução como root.	Container comprometido e risco de movimentação lateral.	Trivy, Dockerfile seguro e usuário não-root.
Permissões excessivas	ServiceAccount com privilégios amplos no cluster.	Escalada de privilégios em Kubernetes.	RBAC e princípio do menor privilégio.
Manipulação de telemetria	Pacotes ambientais chegam sem validação de origem ou integridade.	Forecast incorreto e decisão pública equivocada.	Logs auditáveis, autenticação e validação de integridade.

4. Controles de segurança implementados

Gestão de segredos. O pipeline usa Gitleaks para procurar segredos expostos em commits e pull requests. As credenciais necessárias, como SATELLITE_API_KEY, CLIMATE_API_KEY, MAPS_TOKEN, DATABASE_URL e JWT_SECRET, devem ser cadastradas como GitHub Secrets. Essa escolha evita que tokens sensíveis sejam versionados no repositório ou copiados para imagens Docker.

Análise de código e dependências. O Bandit foi incluído para executar SAST nos microsserviços Python, detectando padrões inseguros no código. O pip-audit foi usado para verificar dependências declaradas no requirements.txt. Esses controles são importantes porque serviços de forecast, geoespacial e saúde pública dependem de bibliotecas externas e podem herdar vulnerabilidades conhecidas.

Segurança em contêineres. O Trivy realiza scan do filesystem e da imagem Docker. O Dockerfile foi escrito com usuário não-root, instalação mínima de dependências e sem armazenamento de credenciais. O Kubernetes deployment inclui securityContext, readOnlyRootFilesystem, bloqueio de privilege escalation, drop de capabilities e limites de CPU/memória.

Zero Trust e mínimo privilégio. O repositório inclui manifestos RBAC para reduzir permissões dos serviços. A lógica é simples: um microsserviço de forecast não precisa permissão administrativa no cluster, e um serviço geoespacial não precisa acessar todos os secrets. Essa separação reduz o impacto caso um componente seja comprometido.

5. Arquitetura DevSecOps do pipeline

Etapa	Controle	Resultado esperado
Commit / Pull Request	GitHub Actions inicia validações	Código só avança se passar nos checks.
Secret Scan	Gitleaks	Bloqueia segredos expostos.
SAST / SCA	Bandit + pip-audit	Detecta código inseguro e dependências vulneráveis.
Docker Build	Dockerfile seguro	Gera imagem com usuário não-root.
Container Scan	Trivy	Bloqueia CVEs críticas ou altas.
Deploy	Kubernetes/K3s	Executa com RBAC e securityContext.
Operação	Logs e alertas	Permite auditoria e resposta a falhas.

6. Implementação prática no repositório

A implementação foi organizada como um projeto GitHub realista. O repositório contém workflow de CI/CD, serviços FastAPI, Dockerfile, docker-compose, manifestos Kubernetes, arquivos de segurança e documentos de evidência. O pipeline não é apenas ilustrativo: ele pode ser executado em GitHub Actions e também possui comandos equivalentes para execução local.

```
name: EcoSaude DevSecOps Pipeline
on: [push, pull_request]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gitleaks/gitleaks-action@v2
      - run: pip install -r requirements.txt bandit pip-audit
      - run: bandit -r services -ll
      - run: pip-audit -r requirements.txt
      - run: docker build -t ecosauade-api:ci .
      - uses: aquasecurity/trivy-action@master
```

A estrutura do repositório também demonstra a integração entre desenvolvimento e segurança. A pasta services contém os microserviços da solução; infrastructure contém os recursos para Kubernetes e Terraform; security contém políticas e scripts; e docs reúne relatório, arquitetura e evidências de execução.

7. Simulação prática de incidente

A simulação escolhida representa um erro comum: uma credencial é inserida diretamente no código. Ao abrir um pull request, o Gitleaks identifica o segredo e interrompe o pipeline. O deploy não acontece. A correção consiste em remover o valor do código, criar a variável em GitHub Secrets e alterar o serviço para ler a credencial por variável de ambiente.

Antes - inseguro	Depois - seguro
SATELLITE_API_KEY = "abc123-secret"	SATELLITE_API_KEY = os.getenv("SATELLITE_API_KEY")

Essa simulação é coerente com o EcoSaúde porque a plataforma depende de integrações externas. A exposição de uma chave poderia comprometer APIs climáticas, dados geoespaciais, bancos de dados e serviços de mapas. O controle automatizado reduz o risco de erro humano chegar a produção.

8. Conexão com o projeto e com os ODS

O módulo DevSecOps não é uma entrega separada do EcoSaúde; ele atua como uma camada de confiabilidade da solução. Como a plataforma gera alertas e forecasts usados para apoiar saúde pública, saneamento e sustentabilidade, a proteção do pipeline garante que os componentes que processam essas informações estejam menos expostos a falhas conhecidas.

A conexão com os ODS ocorre principalmente em quatro frentes. O ODS 3 é contemplado porque o EcoSaúde apoia prevenção e saúde pública. O ODS 6 aparece na análise de água e saneamento. O ODS 9 está presente na construção de uma infraestrutura tecnológica segura, resiliente e automatizada. O ODS 11 é atendido pela proposta de fortalecer cidades e comunidades por meio de informações mais confiáveis.

9. Conclusão

A integração de DevSecOps ao EcoSaúde mostra que segurança precisa acompanhar todo o ciclo de desenvolvimento. Em soluções baseadas em dados espaciais, a proteção de credenciais, APIs, dependências, contêineres e infraestrutura é essencial para manter a integridade das informações. O pipeline proposto automatiza controles relevantes e transforma segurança em parte natural do desenvolvimento, aumentando a confiabilidade técnica e social da solução.

Referências

FIAP. Atividade de Cibersegurança - Global Solution 1º Semestre de 2026. Documento fornecido pela disciplina.

WORLD HEALTH ORGANIZATION. Dengue: global situation, surveillance and progress - 2024. Geneva: WHO, 2025.

WHO/UNICEF JOINT MONITORING PROGRAMME. Progress on household drinking water, sanitation and hygiene 2000-2024. Geneva/New York: WHO/UNICEF, 2025.

QNX SOFTWARE SYSTEMS. QNX Neutrino RTOS System Architecture Guide. Ottawa: QNX Software Systems.

YOCTO PROJECT. The Yocto Project: official documentation and technical overview. Linux Foundation.

AQUA SECURITY. Trivy documentation. Aqua Security.

GITLEAKS. Gitleaks documentation. GitHub project documentation.